32-bit Modular Arithmetic and SIMD Vectorization

Duc Vinh Nguyen, Henry Zheng

March 2026

SORBONNE
UNIVERSITÉ

Contents

1 Introduction **2**

2 Fundamental Cost of Modular Reduction **2**

3 Shoup's Modular Multiplication **2**

3.1 Input Constraints . . . 2
3.2 Theorem . . . 3
3.3 Input Hypotheses . . . 3
3.4 Precomputation Logic (Barrett Reduction) . . . 4
3.5 Fundamental Principle . . . 4
3.6 Demonstration . . . 4
3.7 Advantages . . . 5
3.8 SIMD Vectorization Implementation . . . 6
3.8.1 AVX2 . . . 6
3.9 Performance . . . 7
3.9.1 Overall Hypothesis . . . 7
3.9.2 Equipment Details for AMD Architecture . . . 7
3.9.3 Equipment Details for ARM Architecture . . . 8
3.9.4 Choice of Unrolling . . . 8
3.9.5 Choice Between `mullo_epi32` and `mul_epu32` . . . 9
3.9.6 Overall Performance . . . 11
3.9.7 NEON's Performance . . . 13

4 Overall Conclusion **14**

A SIMD Implementations **15**

A.1 AVX2 . . . 15
A.2 AVX-512 . . . 18
A.3 NEON . . . 23

1 Introduction

This project explores and optimizes modular reductions modulo primes $p < 2^{32}$ using SIMD vectorization (AVX2, AVX-512, NEON) along with various techniques and algorithms.

Objective: The aim of this study is to develop an optimized implementation of modular reduction for 32-bit primes, minimizing execution time through SIMD vectorization.

2 Fundamental Cost of Modular Reduction

The naive approach to computing $a \pmod{n}$ is to evaluate:

$$a - \left\lfloor \frac{a}{n} \right\rfloor \cdot n \tag{1}$$

However, on modern processor architectures (Intel, AMD,...), integer division has a significantly higher computational cost compared to basic arithmetic operations such as additions and multiplications.

Since equation (1) requires an integer division, a multiplication and a subtraction operation, it is considered highly inefficient from both a software and hardware perspective.

3 Shoup's Modular Multiplication

Shoup's algorithm is based on precomputation to avoid costly operations.

Algorithm 1: Shoup's modular multiplication algorithm

Input : $A, B \in [0, p[$, we suppose that $p < \beta/2$
precompute $B' = \lfloor B\beta/p \rfloor$

Output: $R = AB \bmod p$

$Q \leftarrow \lfloor AB'/\beta \rfloor$;
$R \leftarrow (AB - Qp) \bmod \beta$;
if $R \geq p$ **then**
 $R \leftarrow R - p$;

3.1 Input Constraints

To ensure the correctness of the modular reduction and prevent register overflow during SIMD operations, the following conditions must be satisfied:

- **Word Size (β):** Set to 2^{32} for 32-bit modular arithmetic
- **Input 1 (A):** An integer defined over 32 bits
- **Input 2 (B):** Integer within the range $[0, p[$
- **Modulus (p):** A prime number defined over 31 bits ($p < 2^{31}$)

3.2 Theorem

Let p be a prime number such that $p < \beta/2$ where $\beta = 2^{32}$. Given an integer A defined over 32 bits and $B \in [0, p)$, let B' be the precomputed value $B' = \lfloor B\beta/p \rfloor$. If the estimated quotient is defined as $Q = \lfloor AB'/\beta \rfloor$, then the intermediate remainder $R = AB - Qp$ satisfies the following condition:

$$0 \leq AB - Qp < 2p$$

3.3 Input Hypotheses

Can p be any integer over 32 bits?
No, one particularly interesting case where Shoup's algorithm will fail is for $p \geq 2^{31}$. Since we know that:

$$0 \leq AB - Qp < 2p$$

If $p \geq 2^{31}$, then $2p \geq 2^{32}$. Therefore, in some cases $AB - Qp$ may exceed 2^{32}. This causes tremendous errors in the calculation because all bits beyond 32 bits are discarded when we reduce modulo β.

Does p necessarily have to be a prime number?
No, the algorithm mainly uses precomputations (Barrett Reduction), optimized for calculation of $a \mod n$ without constraints on n. But for the sake of this project, n is a prime number (p).

Can b be any integer over 32 bits?
No, suppose $b > p$, we can rewrite b as:

$$b = q' \cdot p + r'$$

Thus the calculation for the precomputed value B' is:

$$B' = \left\lfloor \frac{(q' \cdot p + r') \cdot 2^{32}}{p} \right\rfloor$$

$$B' = q' \cdot 2^{32} + \left\lfloor \frac{r' \cdot 2^{32}}{p} \right\rfloor$$

With a focus on 32 bits in this project, B' in the case of $b > p$ cannot be stored within 32 bits and will overflow.

Can a be any integer over 32 bits?
Yes, a can be any integer defined over 32 bits.

3.4 Precomputation Logic (Barrett Reduction)

The precomputed value B' is defined over 32 bits as:

$$B' = \lfloor (B \cdot 2^{32})/p \rfloor$$

When computing $A \cdot B \pmod p$:

$$A \cdot B \pmod p = A \cdot B - \left\lfloor \frac{A \cdot B}{p} \right\rfloor \cdot p$$

We can rewrite this:

$$\begin{aligned} A \cdot B \pmod p &= A \cdot B - \left\lfloor \frac{A \cdot \left\lfloor (\frac{B \cdot 2^{32}}{p}) \right\rfloor}{2^{32}} \right\rfloor \cdot p \\ &\approx A \cdot B - \left\lfloor \frac{A \cdot B'}{2^{32}} \right\rfloor \cdot p \end{aligned}$$

3.5 Fundamental Principle

Given x, the quotient of two 32-bit integers:
The proof of this algorithm is based on the non-decimal value of ($\lfloor x \rfloor$):

$$x - 1 < \lfloor x \rfloor \leq x$$

We can rewrite this as:

$$0 \leq x - \lfloor x \rfloor < 1$$

3.6 Demonstration

The algorithm uses the precomputation $B' = \lfloor B\beta/p \rfloor$ such that:

$$0 \leq \frac{B\beta}{p} - \left\lfloor \frac{B\beta}{p} \right\rfloor < 1$$

$$0 \leq \frac{B\beta}{p} - B' < 1 \tag{2}$$

Using the same logic, the algorithm estimates the quotient $Q = \lfloor AB'/\beta \rfloor$ such that:

$$0 \leq \frac{AB'}{\beta} - \left\lfloor \frac{AB'}{\beta} \right\rfloor < 1$$

$$0 \leq \frac{AB'}{\beta} - Q < 1 \tag{3}$$

We now prove that the remainder $R = (A \cdot B - Q \cdot p)$ lies within the interval $[0, 2p[$:

We first multiply the equation (1) by Ap/β:

$$0 \le \left(\frac{B\beta}{p} - B'\right) \cdot \frac{Ap}{\beta} < 1 \cdot \frac{Ap}{\beta}$$

$$0 \le AB - \frac{AB'p}{\beta} < \frac{Ap}{\beta} \tag{4}$$

Similarly, we multiply the equation (2) by p:

$$0 \le \left(\frac{AB'}{\beta} - Q\right) \cdot p < 1 \cdot p$$

$$0 \le \frac{AB'p}{\beta} - Qp < p \tag{5}$$

By adding equations (3) and (4), we deduce:

$$0 \le (AB - \cancel{\frac{AB'p}{\beta}}) + (\cancel{\frac{AB'p}{\beta}} - Qp) < \frac{Ap}{\beta} + p$$

Simplifying this result, we are left with:

$$0 \le AB - Qp < p + \frac{Ap}{\beta}$$
$$0 \le AB - Qp < p \cdot (1 + \frac{A}{\beta})$$

By definition, $A \in [0, p[$ and $p < \beta$, thus $A < \beta$. Therefore we can write:

$$0 \le AB - Qp < p \cdot (1 + \frac{A}{\beta}) < p \cdot (1 + \frac{\beta}{\beta})$$
$$0 \le AB - Qp < p \cdot (1 + 1)$$
$$0 \le AB - Qp < 2p$$
$$0 \le R < 2p$$

Conclusion: We have now proven that $R \in [0, 2p[$. The final if statement of the algorithm helps correct the remainder: if $R \ge p$, one more step $R \leftarrow R - p$ is needed; if not, R is the correct result.

3.7 Advantages

Using the precomputed value B' to approximately calculate $\left\lfloor \frac{A \cdot B}{p} \right\rfloor$ improves upon the naive method in two ways:

1. **Storing the precomputed value:** B' is computed with one division and can be stored for use in other modular reductions, provided the same B and the same p are used.

2. **Efficient operations:** Shoup's algorithm replaces slow division operations with fast multiplications and bit shifts. The expression $\frac{A \cdot B'}{2^{32}}$ requires only one multiplication operation and a right bit shift by 32 bits. On a CPU, both multiplication ($\approx$ 0.33–0.67 cycles per operation) and bit shift ($\approx$ 0.5–1 cycles per operation) are significantly more efficient than division ($\approx$ 6 cycles per operation). A division is approximately 4–8 times more costly than a multiplication and bit shift together.

3.8 SIMD Vectorization Implementation

We decided to create three versions for implementing a high-performance Shoup algorithm: AVX2, AVX-512 and NEON.

3.8.1 AVX2

For the AVX2 version, we decided to use two main built-in AVX2 functions:

- `_mm256_mul_epu32(...)`
- `_mm256_mullo_epi32(...)`

Intel `_mm256_mul_epu32(...)` version: (Listings 1)
This function treats the input as unsigned 32-bit integers. However, it only multiplies the even-indexed slots (0, 2, 4, 6) of the vector to make room for the full 64-bit results.

This implementation uses `_mm256_mul_epu32(...)` at all stages of calculation, so we must shuffle the data registers into their correct positions. The following are the steps per loop:

- Load 8 32-bit integers into the register
- Split the loaded register into even-indexed slots (0, 2, 4, 6) and odd-indexed slots (1, 3, 5, 7) into 64-bit registers
- Compute Shoup's modular reduction (`_shoup_avx2(...)`) over these registers
- Recombine the even and odd indexed slots and store
- Compute the leftover elements

Intel `_mm256_mullo_epi32(...)` version 1: (Listings 2)
This function treats the input as signed 32-bit integers. It multiplies all 8 pairs of numbers but only stores the bottom half of the result.

This implementation uses `_mm256_mul_epu32(...)` to compute the quotient:

$$Q = \lfloor (A \cdot B')/2^{32} \rfloor$$

but `_mm256_mullo_epi32()` to compute all other steps per loop. The following are the steps per loop:

- Load 8 32-bit integers into the register
- Split the loaded register into even-indexed slots (0, 2, 4, 6) and odd-indexed slots (1, 3, 5, 7) into 64-bit registers
- Compute Shoup's modular reduction (`_shoup_mullo_avx2(...)`) over these registers

- Recombine the even and odd indexed slots and store
- Compute the leftover elements

Intel `_mm256_mullo_epi32(...)` version 2: (Listings 3)
Derived from the previous version, this version aims to regroup the vector registers immediately after the multiplication step:

$$Q = \lfloor (A \cdot B')/2^{32} \rfloor$$

By restoring the standard 8-lane 32-bit layout early, the function eliminates the need to perform parallel logic paths. This, in theory, should reduce the workload.

In order to implement this version, we create an auxiliary function `_mulhi_emul_avx2(...)` which emulates a native "multiply-high" instruction for unsigned 32-bit integers. It computes the full 64-bit products of two packed vectors, extracts the most significant 32 bits from each result, and repacks them into a single 256-bit destination register.

The following are the steps per loop:

- Load 8 32-bit integers into the register
- Split the loaded register into even-indexed slots (0, 2, 4, 6) and odd-indexed slots (1, 3, 5, 7) into 64-bit registers to compute the quotient
- Recombine the even and odd indexed slots
- Continue with Shoup's modular reduction (`_shoup_mullo_v2_avx2(...)`)
- Compute the leftover elements

3.9 Performance

The performance benchmarks were run on AMD and ARM architectures, by executing `./pcca7 -b_bits 31 -pts 1000`.

3.9.1 Overall Hypothesis

With a rough preliminary analysis, using SIMD operations should provide us with a maximum acceleration of 8x–16x since the registers can hold and operate on 8 32-bit unsigned integers (for AVX2) and 16 32-bit unsigned integers (for AVX-512) simultaneously.

One caveat that must be considered is that all implementations use `_mm256_mul_epu32(...)` to calculate the quotient defined as $Q = \lfloor A \cdot B'/\beta \rfloor$. This operation requires a data shuffle to correctly align the vectors into 64-bit lanes and then recombine them at later stages. This reduces the acceleration factor of SIMD implementations, at worst, by half (4x–8x depending on AVX2 or AVX-512).

3.9.2 Equipment Details for AMD Architecture

- Processor: AMD Ryzen™ 5 8600G (Zen 4)
- Instruction Sets Supported: AVX, AVX2 and AVX-512
- Compiler: GCC 13.3.0 with optimization flag `-O3` and -march=native

3.9.3 Equipment Details for ARM Architecture

- Processor: Apple M1
- Instruction Sets Supported: NEON
- Compiler: GCC 13.3.0 with optimization flag `-O3`

Note: The benchmarks were run in a virtual machine.

3.9.4 Choice of Unrolling

One way to optimize for faster functions through vectorization is loop unrolling. This technique consists of expanding the body of a loop to process multiple iterations at once.

The results below were obtained by measuring the execution time between a standard version and an unrolled version of Listing 2.

Hypothesis: The unrolled version should perform better than the standard version.

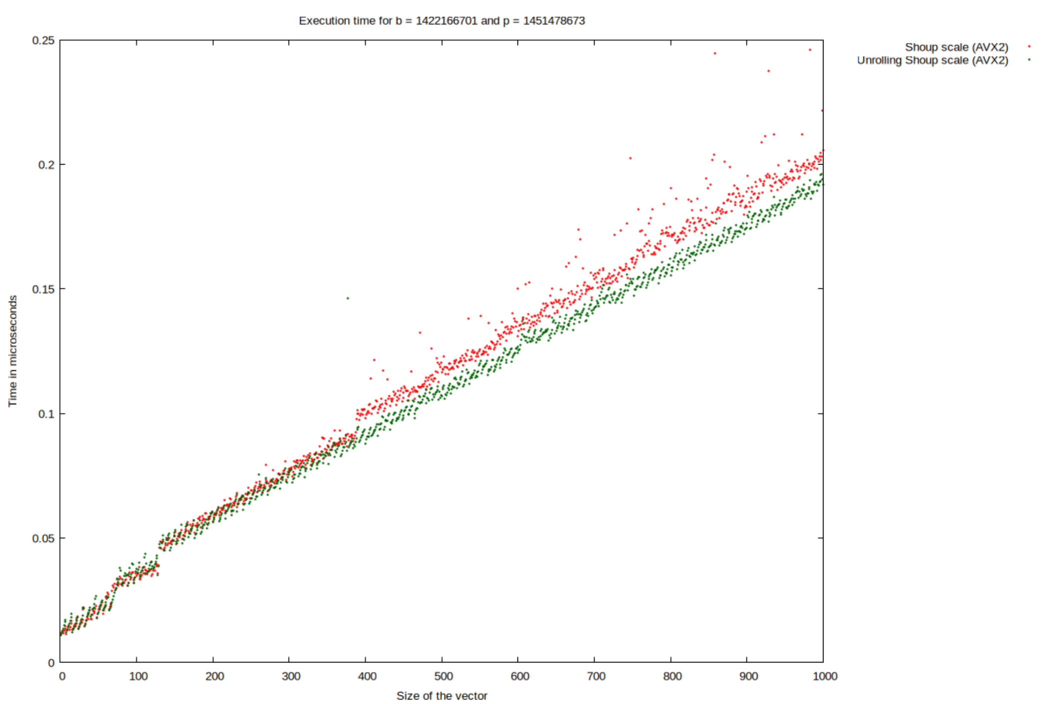

Figure 1: Execution time comparison: Standard vs. Unrolled

Size	Standard	Unrolled	Percentage difference
100	0.034	0.037	+8.82%
200	0.056	0.056	+0.00%
300	0.076	0.076	+0.00%
400	0.098	0.090	−8.12%
500	0.116	0.107	−7.76%
600	0.131	0.124	−5.34%
700	0.154	0.142	−7.80%
800	0.190	0.155	−18.42%
900	0.188	0.176	−6.38%
1000	0.205	0.191	−6.83%

Table 1: Execution time of implementations with and without unrolling in microseconds

Observation: On average, the unrolled implementation is approximately 5% faster than the standard version.

Conclusion: The results obtained align with our preliminary assumption that applying unrolling to the loops will increase performance. Although the speedup is only around 5% (x1.05 speedup factor), we can still conclude that unrolling is favorable for our implementation of Shoup's modular multiplication

3.9.5 Choice Between `mullo_epi32` and `mul_epu32`

`mullo_epi32` and `mul_epu32` both produce at most 64-bit results. The key difference is that the former truncates the result and stores only the lower 32 bits, whereas the latter stores the full 64-bit result.

The results below were obtained by measuring the execution time between the three variations with unrolling described in Section 3.8.1.

Hypothesis: The two `mullo_epi32` variants should perform better than the `mul_epu32` variant because they use fewer operations overall. The `mullo_epi32` (v2) variant (Listing 3) should perform better than the `mullo_epi32` (v1) variant (Listing 2).

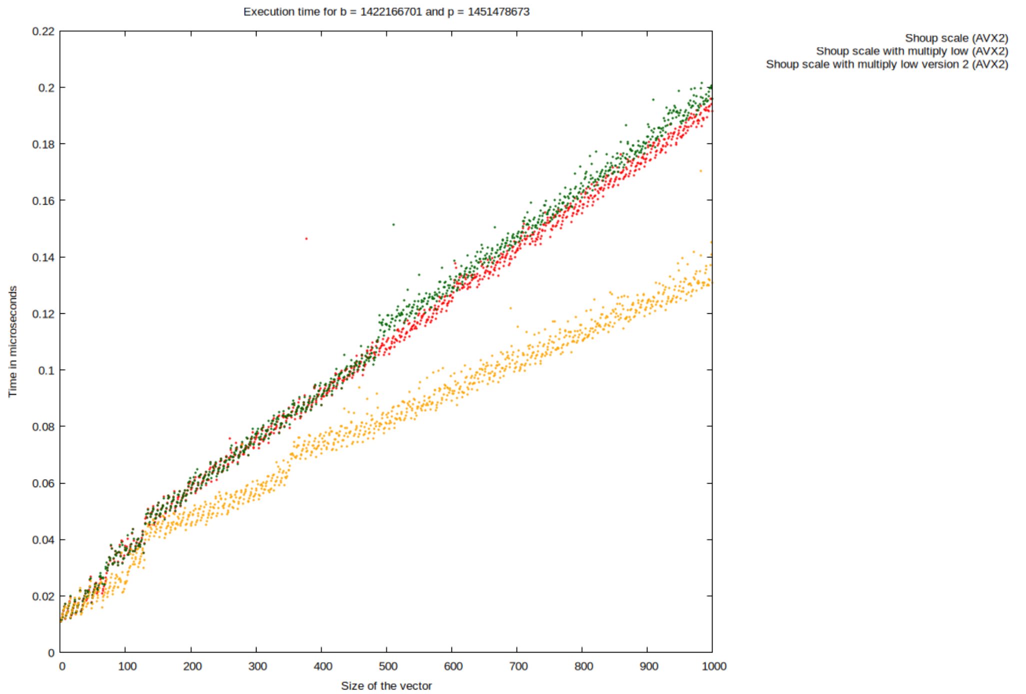

Figure 2: Execution time comparison: mullo_epi32 vs. mul_epu32

Size	mul_epu32	mullo_epi32 (v1)	x	mullo_epi32 (v2)	x
100	0.037	0.037	+0.00%	0.024	−35.13%
200	0.056	0.056	+0.00%	0.045	−19.64%
300	0.076	0.077	+1.32%	0.057	−25.00%
400	0.090	0.087	−3.33%	0.070	−22.22%
500	0.107	0.116	+8.41%	0.081	−24.29%
600	0.124	0.127	+2.41%	0.089	−28.22%
700	0.142	0.147	+3.52%	0.102	−28.17%
800	0.155	0.161	+3.87%	0.113	−27.10%
900	0.176	0.182	+3.40%	0.123	−30.11%
1000	0.191	0.196	+2.61%	0.130	−31.93%

Table 2: Execution time of all implementations and the speedup in comparison to mul_epu32 in microseconds

Observation: From the graph, we can see that for small vector sizes (under 500), the performance difference between `mullo_epi32` (v1) and `mul_epu32` is negligible. However, the efficiency gain starts to appear once the vector size reaches 500 and above.

As for `mullo_epi32` (v2), it outperforms both previous variants for all sizes.

On average, `mullo_epi32` (v1) is 2.22% slower than `mul_epu32`, whereas `mullo_epi32` (v2) is 24.85% faster than `mul_epu32`.

Conclusion: The measured results contradict our hypothesis that both `mullo_epi32` variants are faster than `mul_epu32`. Although `mullo_epi32` (v1) is a worse implementation of Shoup's modular multiplication, we can still conclude that `mullo_epi32` (v2) is the best implementation among the variants, with a performance gain of approximately 25% (x1.25 speedup factor).

3.9.6 Overall Performance

It is worth noting that AVX2 uses 256-bit registers to process up to eight 32-bit integers, while AVX-512 uses 512-bit registers for up to sixteen. Furthermore, the AVX-512 version uses the same functions and operations as the AVX2 version.

To measure the best performance output, we decided that the fastest implementation : mullo_epi32 version 2 (see Listing 3) with unrolling

The results below were obtained by measuring the execution time of different implementations of modular reduction:

- **Naive scale:** The base operation (%)
- **Shoup scale (reference) :** Shoup's modular multiplication without optimization
- **Shoup scale (FLINT) :** FLINT's modular reduction implementation
- **Shoup scale (AVX2) :** Shoup's modular multiplication with AVX2 optimization (mullo version 2 with unrolling)
- **Shoup scale (AVX-512) :** Shoup's modular multiplication with AVX-512 optimization (mullo version 2 with unrolling)

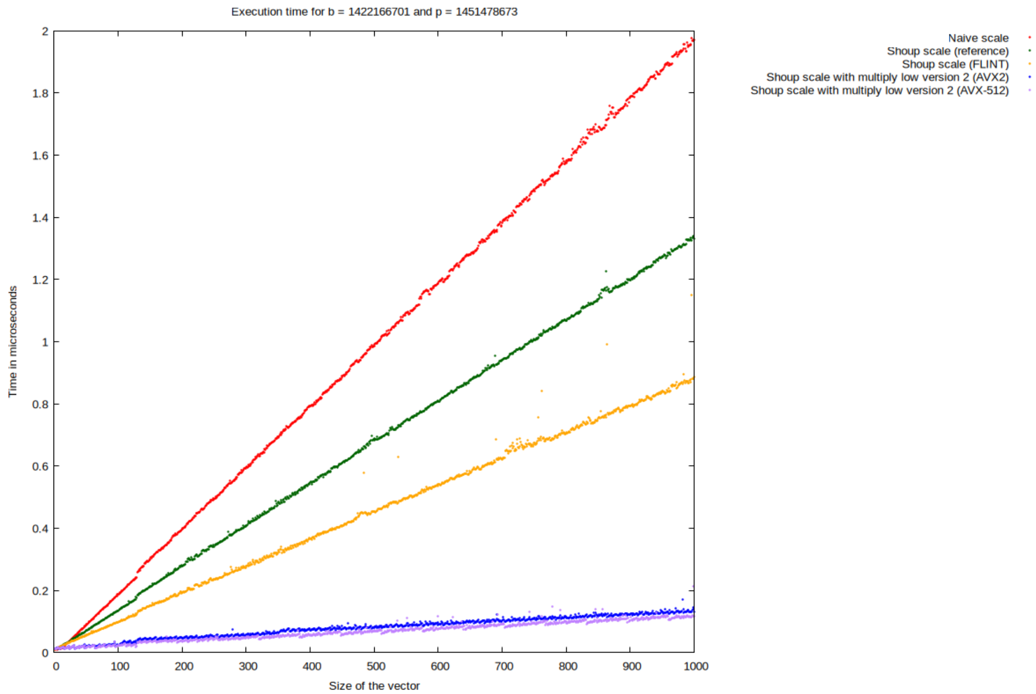

Figure 3: Execution time comparison between different implementations of modular reduction

Size	Naive	Shoup ref	FLINT	Shoup AVX2	Shoup AVX-512
100	0.187	0.135	0.098	0.024	0.022
200	0.397	0.277	0.193	0.045	0.035
300	0.594	0.408	0.279	0.056	0.048
400	0.794	0.546	0.365	0.070	0.053
500	0.986	0.687	0.454	0.081	0.068
600	1.186	0.806	0.541	0.089	0.075
700	1.382	0.940	0.621	0.102	0.091
800	1.574	1.068	0.709	0.113	0.094
900	1.786	1.198	0.794	0.123	0.101
1000	1.971	1.330	0.886	0.130	0.116

Table 3: Execution time for different implementations in relation to the size of the vector in milliseconds

Algorithm	Average %	Factor of acceleration
FLINT	100.00%	1x
Shoup (AVX2)	16.86%	5.9x
Shoup (AVX-512)	14.35%	7.0x

Table 4: Average performance summary of FLINT in relation to Shoup AVX2 and Shoup AVX-512 implementations

Observation: From the results retrieved, we want to primarily study the speedup factor of our Shoup's modular multiplication using AVX2 and AVX-512 compared to FLINT's implementation. Shoup (AVX2) is roughly 5.9x faster than FLINT and Shoup (AVX-512) is 7.0x faster.

Conclusion: Our Shoup's modular multiplication with AVX2 and AVX-512 optimizations adapted to 32 bits is 5–7 times faster than FLINT. These results are consistent with our overall hypothesis.

3.9.7 NEON's Performance

Unlike AVX2 and AVX-512, which use opaque data types, NEON uses explicit vector types: each NEON type name defines the data type, the number of bits, and the number of elements.

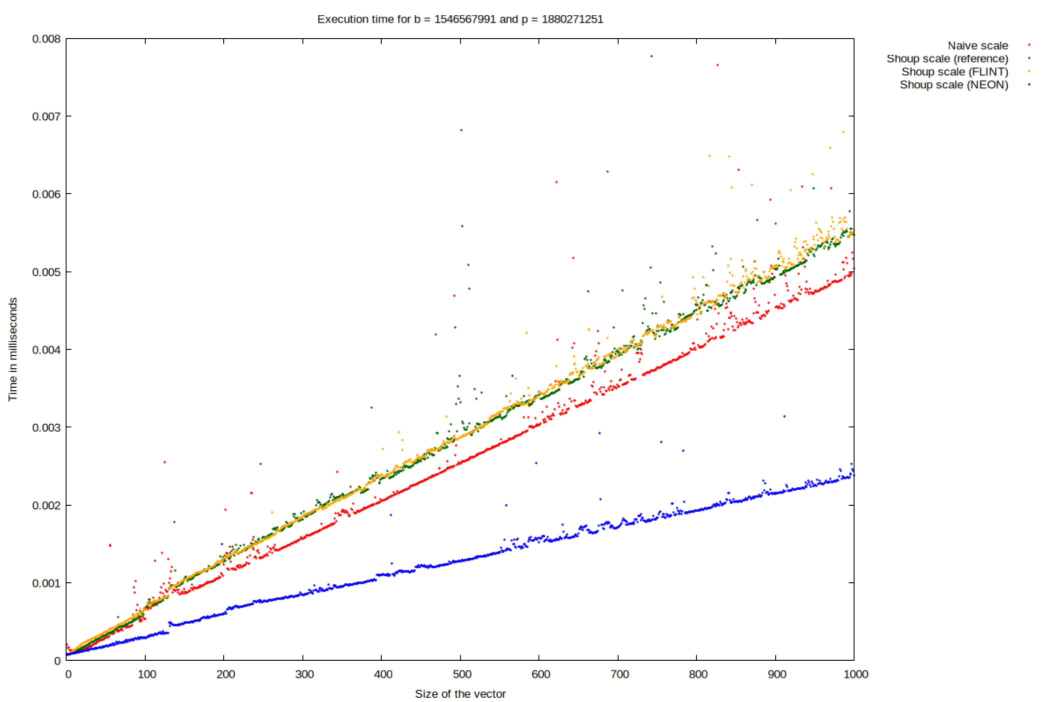

Figure 4: Execution time comparison between the two versions of multiply low

4 Overall Conclusion

For AVX architectures, we researched and implemented several versions of Shoup's modular reduction. The best-performing version is **Intel `_mm256_mullo_epi32(...)` version 2**, with an acceleration factor of 5.9x for AVX2 and 7.0x for AVX-512.

A SIMD Implementations

A.1 AVX2

Listing 1: Auxiliary function of Shoup's modular reduction

```
static inline __m256i _shoup_avx2(__m256i va,
                                  __m256i vb,
                                  __m256i vb_precomp,
                                  __m256i vp)
{
    // vq = [ (a_0 * b_precomp_0 = q_0q_1), (a_2 * b_precomp_2 = q_2q_3),
    // (a_4 * b_precomp_4 = q_4q_5), (a_6 * b_precomp_6 = q_6q_7) ]
    __m256i vq = _mm256_mul_epu32(va, vb_precomp);

    // vq = [ (0, q_0), (0, q_2), (0, q_4), (0, q_6) ]
    vq = _mm256_srli_epi64(vq, 32);

    // vab = [ (a_0 * b_0 = ab_0ab_1), (a_2 * b_2 = ab_2ab_3),
    // (a_4 * b_4 = ab_4ab_5), (a_6 * b_6 = ab_6ab_7) ]
    __m256i vab = _mm256_mul_epu32(va, vb);

    // vqp = [ (q_0 * p_0 = qp_0qp_1), (q_2 * p_2 = qp_2qp_3),
    // (q_4 * p_4 = qp_4qp_5), (q_6 * p_6 = qp_6qp_7) ]
    __m256i vqp = _mm256_mul_epu32(vq, vp);

    // vc = [ (ab_0 - qp_0 = c_0c_1), (ab_2 - qp_2 = c_2c_3),
    // (ab_4 - qp_4 = c_4c_5), (ab_6 - qp_6 = c_6c_7) ]
    __m256i vc = _mm256_sub_epi64(vab, vqp);

    // if (c >= p) c -= p
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}

Vector shoup_scale_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);
    ulong i = 0;
    for (; i + 7 < size; i += 8)
    {
        // Load [ a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7 ]
        //    = [ (a_1, a_0), (a_3, a_2), (a_5, a_4), (a_7, a_6) ]
        // but only even index will be directly handled
        __m256i even_va = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i)
        );

        // Need to move odd index to even index to be handled:
        // [ (0, a_1), (0, a_3), (0, a_5), (0, a_7) ]
        __m256i odd_va = _mm256_srli_epi64(even_va, 32);

        // Process Even Lanes
        __m256i even_vc = _shoup_avx2(even_va, vb, vb_precomp, vp);

        // Process Odd Lanes:
        __m256i odd_vc = _shoup_avx2(odd_va, vb, vb_precomp, vp);
```

```

        // Shift odd by 32 bits to the left results back and merge:
        // [ (c_1, 0), (c_3, 0), (c_5, 0), (c_7, 0) ]
        __m256i odd_shifted = _mm256_slli_epi64(odd_vc, 32);

        // Merge even and odd: [ c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7 ]
        __m256i merge = _mm256_or_si256(even_vc, odd_shifted);

        _mm256_storeu_si256((__m256i *)(res.elements + i), merge);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 2: Auxiliary function of Shoup's modular reduction

```
static inline __m256i _shoup_mullo_avx2(__m256i va,
                                        __m256i vb,
                                        __m256i vb_precomp,
                                        __m256i vp)
{
    // vq = [ (a_0 * b_precomp_0 = q_0q_1), (a_2 * b_precomp_2 = q_2q_3),
    // (a_4 * b_precomp_4 = q_4q_5), (a_6 * b_precomp_6 = q_6q_7) ]
    __m256i vq = _mm256_mul_epu32(va, vb_precomp);

    // vq = [ (0, q_0), (0, q_2), (0, q_4), (0, q_6) ]
    vq = _mm256_srli_epi64(vq, 32);

    // vab = [ (a_0 * b_0 = ab_0ab_1), (a_2 * b_2 = ab_2ab_3),
    // (a_4 * b_4 = ab_4ab_5), (a_6 * b_6 = ab_6ab_7) ]
    __m256i vab = _mm256_mullo_epi32(va, vb);

    // vqp = [ (q_0 * p_0 = qp_0qp_1), (q_2 * p_2 = qp_2qp_3),
    // (q_4 * p_4 = qp_4qp_5), (q_6 * p_6 = qp_6qp_7) ]
    __m256i vqp = _mm256_mullo_epi32(vq, vp);

    // vc = [ (ab_0 - qp_0 = c_0c_1), (ab_2 - qp_2 = c_2c_3),
    // (ab_4 - qp_4 = c_4c_5), (ab_6 - qp_6 = c_6c_7) ]
    __m256i vc = _mm256_sub_epi64(vab, vqp);

    // if (c >= p) c -= p
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}

Vector shoup_scale_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);
    ulong i = 0;
    for (; i + 31 < size; i += 32)
    {
        __m256i even_va_0 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i));
```

```
        __m256i odd_va_0 = _mm256_srli_epi64(even_va_0, 32);

        __m256i even_vc_0 = _shoup_mullo_avx2(even_va_0, vb, vb_precomp, vp);

        __m256i odd_vc_0 = _shoup_mullo_avx2(odd_va_0, vb, vb_precomp, vp);

        __m256i odd_shifted_0 = _mm256_slli_epi64(odd_vc_0, 32);

        __m256i merge_0 = _mm256_or_si256(even_vc_0, odd_shifted_0);

        _mm256_storeu_si256((__m256i *)(res.elements + i), merge_0);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 3: Auxiliary function `_mulhi_emul_avx2(...)`

```
static inline m256i _mulhi_emul_avx2(m256i va, m256i vb)
{
    // res_even = [ (a_0 * b_0), (a_2 * b_2), (a_4 * b_4), (a_6 * b_6) ]
    m256i res_even = _mm256_mul_epu32(va, vb);

    // res_odd = [ (a_1 * b_0), (a_3 * b_2), (a_5 * b_4), (a_7 * b_6) ]
    m256i res_odd = _mm256_mul_epu32(_mm256_srli_epi64(va, 32), vb);

    // hi_even = [ (0, hi(a_0 * b_0)), (0, hi(a_2 * b_2)), (0, hi(a_4 * b_4)),
    // (0, hi(a_6*b_6)) ]
    m256i hi_even = _mm256_srli_epi64(res_even, 32);

    // Removes the least significant bits: hi_odd = [ (hi(a_1 * b_0), 0),
    // (hi(a_3 * b_2), 0), (hi(a_5 * b_4), 0), (hi(a_7 * b_6), 0) ]
    __m256i hi_odd = _mm256_and_si256(res_odd, _mm256_set1_epi64x(
        0xFFFFFFFF00000000ULL));

    // [ (hi(a_1 * b_0), hi(a_0 * b_0)), (hi(a_3 * b_2), hi(a_2 * b_2)),
    // (hi(a_5 * b_4), hi(a_4 * b_4)), (hi(a_7 * b_6), hi(a_6 * b_6)) ]
    return _mm256_or_si256(hi_even, hi_odd);
}

static inline m256i _shoup_mullo_v2_avx2(m256i va,
                                         m256i vb,
                                         m256i vb_precomp,
                                         m256i vp)
{
    m256i vq = _mulhi_emul_avx2(va, vb_precomp);
    m256i vab = _mm256_mullo_epi32(va, vb);
    m256i vqp = _mm256_mullo_epi32(vq, vp);
    __m256i vc = _mm256_sub_epi32(vab, vqp);
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}

Vector shoup_scale_mullo_v2_avx2(Parameters param, Vector v)
```

```
{
    ulong size = v.size;
    Vector res = init_vector(size);
    m256i vb = _mm256_set1_epi32(param.b);
    m256i vb_precomp = _mm256_set1_epi32(param.b_precomp);
    m256i vp = _mm256_set1_epi32(param.p);
    ulong i = 0;
    for (; i + 7 < size; i += 8)
    {
        m256i va = _mm256_loadu_si256((m256i const *)(v.elements + i));

        m256i vc = _shoup_mullo_v2_avx2(va, vb, vb_precomp, vp);

        _mm256_storeu_si256((__m256i *)(res.elements + i), vc);
    }
    for (; i < size; i++)
        (res.elements + i) = shoup((v.elements + i),
                                   param.b,
                                   param.b_precomp,
                                   param.p);
    return res;
}
```

A.2 AVX-512

Listing 4: Auxiliary function of Shoup's modular reduction

```
static inline __m512i _shoup_avx512(__m512i va,
                                    __m512i vb,
                                    __m512i vb_precomp,
                                    __m512i vp)
{
    __m512i vq = _mm512_mul_epu32(va, vb_precomp);
    vq = _mm512_srli_epi64(vq, 32);
    __m512i vab = _mm512_mul_epu32(va, vb);
    __m512i vqp = _mm512_mul_epu32(vq, vp);
    __m512i vc = _mm512_sub_epi64(vab, vqp);
    return _mm512_min_epu32(vc, _mm512_sub_epi32(vc, vp));
}
```

Listing 5: Implementation of Shoup's modular reduction

```
Vector shoup_scale_avx512(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m512i vb = _mm512_set1_epi64(param.b);
    __m512i vb_precomp = _mm512_set1_epi64(param.b_precomp);
    __m512i vp = _mm512_set1_epi64(param.p);
    ulong i = 0;
    for (; i + 15 < size; i += 16)
    {
        __m512i even_va = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i));

        __m512i odd_va = _mm512_srli_epi64(even_va, 32);

        __m512i even_vc = _shoup_avx512(even_va, vb, vb_precomp, vp);

```

```
        __m512i odd_vc = _shoup_avx512(odd_va, vb, vb_precomp, vp);

        __m512i odd_shifted = _mm512_slli_epi64(odd_vc, 32);

        __m512i merge = _mm512_or_si512(even_vc, odd_shifted);

        _mm512_storeu_si512((__m512 *)(res.elements + i), merge);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 6: Implementation of unrolling Shoup's modular reduction

```
Vector unrolling_shoup_scale_avx512(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m512i vb = _mm512_set1_epi64(param.b);
    __m512i vb_precomp = _mm512_set1_epi64(param.b_precomp);
    __m512i vp = _mm512_set1_epi64(param.p);
    ulong i = 0;
    for (; i + 63 < size; i += 64)
    {
        __m512i even_va_0 = _mm512_loadu_si512((__m512i const *)(v.elements
            + i));
        __m512i even_va_1 = _mm512_loadu_si512((__m512i const *)(v.elements
            + i + 16));
        __m512i even_va_2 = _mm512_loadu_si512((__m512i const *)(v.elements
            + i + 32));
        __m512i even_va_3 = _mm512_loadu_si512((__m512i const *)(v.elements
            + i + 48));

        __m512i odd_va_0 = _mm512_srli_epi64(even_va_0, 32);
        __m512i odd_va_1 = _mm512_srli_epi64(even_va_1, 32);
        __m512i odd_va_2 = _mm512_srli_epi64(even_va_2, 32);
        __m512i odd_va_3 = _mm512_srli_epi64(even_va_3, 32);

        __m512i even_vc_0 = _shoup_avx512(even_va_0, vb, vb_precomp, vp);
        __m512i even_vc_1 = _shoup_avx512(even_va_1, vb, vb_precomp, vp);
        __m512i even_vc_2 = _shoup_avx512(even_va_2, vb, vb_precomp, vp);
        __m512i even_vc_3 = _shoup_avx512(even_va_3, vb, vb_precomp, vp);

        __m512i odd_vc_0 = _shoup_avx512(odd_va_0, vb, vb_precomp, vp);
        __m512i odd_vc_1 = _shoup_avx512(odd_va_1, vb, vb_precomp, vp);
        __m512i odd_vc_2 = _shoup_avx512(odd_va_2, vb, vb_precomp, vp);
        __m512i odd_vc_3 = _shoup_avx512(odd_va_3, vb, vb_precomp, vp);

        __m512i odd_shifted_0 = _mm512_slli_epi64(odd_vc_0, 32);
        __m512i odd_shifted_1 = _mm512_slli_epi64(odd_vc_1, 32);
        __m512i odd_shifted_2 = _mm512_slli_epi64(odd_vc_2, 32);
        __m512i odd_shifted_3 = _mm512_slli_epi64(odd_vc_3, 32);

        __m512i merge_0 = _mm512_or_si512(even_vc_0, odd_shifted_0);
        __m512i merge_1 = _mm512_or_si512(even_vc_1, odd_shifted_1);
        __m512i merge_2 = _mm512_or_si512(even_vc_2, odd_shifted_2);
```

```c
        __m512i merge_3 = _mm512_or_si512(even_vc_3, odd_shifted_3);

        _mm512_storeu_si512((__m512 *)(res.elements + i), merge_0);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 16), merge_1);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 32), merge_2);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 48), merge_3);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 7: Auxiliary function of Shoup's modular reduction (version with multiply-low)

```c
static inline __m512i _shoup_mullo_avx512(__m512i va,
                                          __m512i vb,
                                          __m512i vb_precomp,
                                          __m512i vp)
{
    __m512i vq = _mm512_mul_epu32(va, vb_precomp);
    vq = _mm512_srli_epi64(vq, 32);
    __m512i vab = _mm512_mullo_epi32(va, vb);
    __m512i vqp = _mm512_mullo_epi32(vq, vp);
    __m512i vc = _mm512_sub_epi32(vab, vqp);
    return _mm512_min_epu32(vc, _mm512_sub_epi32(vc, vp));
}
```

Listing 8: Implementation of Shoup's modular reduction (version with multiply-low)

```c
Vector shoup_scale_mullo_avx512(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m512i vb = _mm512_set1_epi64(param.b);
    __m512i vb_precomp = _mm512_set1_epi64(param.b_precomp);
    __m512i vp = _mm512_set1_epi64(param.p);
    ulong i = 0;
    for (; i + 63 < size; i += 64)
    {
        __m512i even_va_0 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i));
        __m512i even_va_1 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i + 16));
        __m512i even_va_2 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i + 32));
        __m512i even_va_3 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i + 48));

        __m512i odd_va_0 = _mm512_srli_epi64(even_va_0, 32);
        __m512i odd_va_1 = _mm512_srli_epi64(even_va_1, 32);
        __m512i odd_va_2 = _mm512_srli_epi64(even_va_2, 32);
        __m512i odd_va_3 = _mm512_srli_epi64(even_va_3, 32);

        __m512i even_vc_0 = _shoup_mullo_avx512(even_va_0,
                                                vb,
                                                vb_precomp,
                                                vp);
```

```
        __m512i even_vc_1 = _shoup_mullo_avx512(even_va_1,
                                                vb,
                                                vb_precomp,
                                                vp);
        __m512i even_vc_2 = _shoup_mullo_avx512(even_va_2,
                                                vb,
                                                vb_precomp,
                                                vp);
        __m512i even_vc_3 = _shoup_mullo_avx512(even_va_3,
                                                vb,
                                                vb_precomp,
                                                vp);

        __m512i odd_vc_0 = _shoup_mullo_avx512(odd_va_0, vb, vb_precomp, vp);
        __m512i odd_vc_1 = _shoup_mullo_avx512(odd_va_1, vb, vb_precomp, vp);
        __m512i odd_vc_2 = _shoup_mullo_avx512(odd_va_2, vb, vb_precomp, vp);
        __m512i odd_vc_3 = _shoup_mullo_avx512(odd_va_3, vb, vb_precomp, vp);

        __m512i odd_shifted_0 = _mm512_slli_epi64(odd_vc_0, 32);
        __m512i odd_shifted_1 = _mm512_slli_epi64(odd_vc_1, 32);
        __m512i odd_shifted_2 = _mm512_slli_epi64(odd_vc_2, 32);
        __m512i odd_shifted_3 = _mm512_slli_epi64(odd_vc_3, 32);

        __m512i merge_0 = _mm512_or_si512(even_vc_0, odd_shifted_0);
        __m512i merge_1 = _mm512_or_si512(even_vc_1, odd_shifted_1);
        __m512i merge_2 = _mm512_or_si512(even_vc_2, odd_shifted_2);
        __m512i merge_3 = _mm512_or_si512(even_vc_3, odd_shifted_3);

        _mm512_storeu_si512((__m512 *)(res.elements + i), merge_0);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 16), merge_1);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 32), merge_2);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 48), merge_3);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 9: Auxiliary function _mulhi_emul_avx512(...)

```
static inline __m512i _mulhi_emul_avx512(__m512i va, __m512i vb)
{
    __m512i res_even = _mm512_mul_epu32(va, vb);
    __m512i res_odd = _mm512_mul_epu32(_mm512_srli_epi64(va, 32), vb);
    __m512i hi_even = _mm512_srli_epi64(res_even, 32);
    __m512i hi_odd = _mm512_and_si512(res_odd,
                                      _mm512_set1_epi64(
                                        0xFFFFFFFF00000000ULL));
    return _mm512_or_si512(hi_even, hi_odd);
}
```

Listing 10: Auxiliary function of Shoup's modular reduction (version 2 of multiply-low)

```
static inline __m512i _shoup_mullo_v2_avx512(__m512i va,
                                             __m512i vb,
                                             __m512i vb_precomp,
                                             __m512i vp)
```

```
{
    __m512i vq = _mulhi_emul_avx512(va, vb_precomp);
    __m512i vab = _mm512_mullo_epi32(va, vb);
    __m512i vqp = _mm512_mullo_epi32(vq, vp);
    __m512i vc = _mm512_sub_epi32(vab, vqp);
    return _mm512_min_epu32(vc, _mm512_sub_epi32(vc, vp));
}
```

Listing 11: Implementation of Shoup's modular reduction (version 2 of multiply-low)

```
Vector shoup_scale_mullo_v2_avx512(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m512i vb = _mm512_set1_epi32(param.b);
    __m512i vb_precomp = _mm512_set1_epi32(param.b_precomp);
    __m512i vp = _mm512_set1_epi32(param.p);
    ulong i = 0;
    for (; i + 15 < size; i += 16)
    {
        __m512i va = _mm512_loadu_si512((__m512i const *)(v.elements + i));

        __m512i vc = _shoup_mullo_v2_avx512(va, vb, vb_precomp, vp);
        _mm512_storeu_si512((__m512i *)(res.elements + i), vc);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

A.3 NEON

Listing 12: Auxiliary function of Shoup's modular reduction

```
static inline uint32x2_t _shoup_neon(uint32x2_t va,
                                     uint32x2_t vb,
                                     uint32x2_t vb_precomp,
                                     uint32x2_t vp)
{
    // vab_precomp = [ a_0 * b_precomp_0 = ab_precomp_0,
    // a_1 * b_precomp_1 = ab_precomp_1 ]
    uint64x2_t vab_precomp = vmull_u32(va, vb_precomp);

    // vq = [ q_0, q_1 ]
    uint32x2_t vq = vshrn_n_u64(vab_precomp, 32);

    // vab = [ a_0 * b_0 = ab_0, a_1 * b_1 = ab_1 ]
    uint64x2_t vab = vmull_u32(va, vb);

    // vqp = [ q_0 * p_0 = qp_0, q_1 * p_1 = qp_1 ]
    uint64x2_t vqp = vmull_u32(vq, vp);

    // vc = [ ab_0 - qp_0 = c_0, ab_1 - qp_1 = c_1 ]
    uint32x2_t vc = vmovn_u64(vsubq_u64(vab, vqp));

    // if (c >= p) c -= p
    return vmin_u32(vc, vsub_u32(vc, vp));
}
```

Listing 13: NEON Implementation of Shoup's modular reduction

```
Vector shoup_scale_neon(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    ulong n = size - (size % 4);
    uint32x2_t vb = vdup_n_u32(param.b);
    uint32x2_t vp = vdup_n_u32(param.p);
    uint32x2_t vb_precomp = vdup_n_u32(param.b_precomp);
    ulong i = 0;
    for (; i + 1 < n; i += 2)
    {
        // Load [ a_0, a_1 ]
        uint32x2_t va = vld1_u32(v.elements + i);

        // Store the value
        vst1_u32(res.elements + i, _shoup_neon(va, vb, vb_precomp, vp));
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 14: Auxiliary function of Shoup's modular reduction (Version with multiply-low)

```
static inline uint32x2_t _shoup_mullo_neon(uint32x2_t va,
                                           uint32x2_t vb,
```

```
                                            uint32x2_t vb_precomp,
                                            uint32x2_t vp)
{
    // vab_precomp = [ a_0 * b_precomp_0 = ab_precomp_0,
    // a_1 * b_precomp_1 = ab_precomp_1 ]
    uint64x2_t vab_precomp = vmull_u32(va, vb_precomp);

    // vq = [ q_0, q_1 ]
    uint32x2_t vq = vshrn_n_u64(vab_precomp, 32);

    // vab = [ a_0 * b_0 mod 2^32 = ab_0, a_1 * b_1 mod 2^32 = ab_1 ]
    uint32x2_t vab = vmul_u32(va, vb);

    // vqp = [ q_0 * p_0 mod 2^32 = qp_0, q_1 * p_1 mod 2^32 = qp_1 ]
    uint32x2_t vqp = vmul_u32(vq, vp);

    // vc = [ ab_0 - qp_0 = c_0, ab_1 - qp_1 = c_1]
    uint32x2_t vc = vsub_u32(vab, vqp);
    return vmin_u32(vc, vsub_u32(vc, vp));
}
```